

Relatório Técnico — Interface Inteligente Unificada v2

DESAFIO 4: Interface Inteligente para Consulta de Arquivos CSV

****Grupo:** I2A2 InsurTechLab — Versão Final Integrada******

****Data:** Agosto 2026******

****Repositórios base:****

- <https://github.com/InsurTechLabI2A2/Interface-Inteligente-para-Consulta-de-Arquivos-CSV>

- <https://github.com/InsurTechLabI2A2/Multiagente-de-Consulta-de-Nfe-v1>

1. Objetivo

Desenvolver um ou mais agentes inteligentes capazes de responder perguntas em linguagem natural sobre conjuntos de dados armazenados nos arquivos `202401\_NFs` e `202505\_NFe`, disponíveis em https://drive.google.com/drive/folders/19InGHYOQnTou_N0_KAXZ6tK9WB6rD6HS

Requisitos mínimos atendidos:

- Upload de ZIPs contendo CSVs
- Processamento automático
- Interface de perguntas em linguagem natural
- Respostas em texto, tabela ou gráfico
- Pelo menos um framework de agentes (LangChain)
- Separação clara interface/agentes/ferramentas/processamento
- Organização em módulos, prompts claros, tratamento de erros, ocultar chaves
- Relatório técnico + 10 perguntas + código-fonte + GitHub Pages

2. Framework Escolhido

****LangChain**** (obrigatório) + ****Pydantic**** para saída estruturada. Motivo: permite criar agente com `create\_agent`, definir contrato `QueryPlan` validado, e ter fallback determinístico offline. Para GitHub Pages, usa-se JS puro com mesma lógica de agentes, carregando LangChain apenas quando `ANTHROPIC\_API\_KEY` está disponível em `.env` (Python) ou sandbox Claude.ai (JS tenta chamar Claude API, se falhar cai para heurística).

Ferramentas auxiliares:

- ****JSZip + PapaParse + Chart.js**** via CDN (100% client-side, sem instalação)
- ****Pandas**** para execução determinística
- ****Streamlit**** para interface Python avançada

- **Plotly** opcional para pizza

3. Arquitetura da Solução

Vide `arquitetura.md`. 6 agentes especializados:

1. **LoaderAgent**: valida ZIP, extrai CSVs, detecta encoding, remove linhas vazias.
2. **SchemaAgent**: infere tipos, gera dicionário automático, mapeia sinônimos PT-BR.
3. **QueryAgent**: NLU híbrido — LLM (LangChain) se disponível, senão heurística keyword-based, produz `QueryPlan` Pydantic.
4. **ExecutorAgent**: execução whitelist (sum, avg, count, max, min, group_sum, growth), sem eval.
5. **VizAgent**: decide visualização (text/table/bar/line/pie) e renderiza.
6. **FallbackAgent**: mensagens claras, sugestão de reformulação, lista colunas.

Interface A (Carga) → Interface B (Consulta) → Evidências (logs, dicionário, histórico)

Duas implementações:

- **index.html** (GitHub Pages): tudo client-side, sem backend, acessível online
- **app.py** (Streamlit): para testes locais avançados com LangChain

4. Fluxo de Funcionamento

1. Usuário arrasta ZIPs na área de upload (ou usa dados de exemplo)
2. LoaderAgent extrai CSVs, loga cada arquivo
3. SchemaAgent gera dicionário e exibe KPI (tabelas, linhas, colunas)
4. Usuário digita pergunta ou clica em pergunta de teste
5. QueryAgent gera QueryPlan JSON (mostrado antes da execução se checkbox ativado)
6. ExecutorAgent executa plano sobre DataFrame
7. VizAgent decide melhor visualização e renderiza texto + tabela + gráfico Chart.js
8. Logs técnicos por agente são acumulados e exportáveis
9. FallbackAgent captura qualquer erro e sugere reformulação

5. Como Cada Agente Toma Decisões

Loader: verifica extensão .zip, tenta abrir com JSZip/zipfile, lista arquivos .csv, ignora diretórios. Se nenhum CSV, lança erro capturado por Fallback.

Schema: amostra 100 primeiras linhas, conta quantas são números/datas. >80% números = numérico. Mapeia descrição via dicionário interno (xFant→fornecedor). Decide nome da tabela alvo: prioriza quem tem vNF/vProd, senão maior tabela.

Query:

- LLM: prompt = schema + pergunta + instruções de saída JSON. Valida com Pydantic. Confidence 0.9.
- Heurística: regex PT-BR: "maior valor" → intent ranking metric sum viz bar; "por mês" → timeseries line; "top 5" → limit 5; "quantidade por NCM" → group_count pie. Confidence 0.65.
- Se grupo não encontrado, busca sinônimo; se ainda não, usa primeira coluna.

****Executor****: converte valor BR via regex, escolhe coluna numérica se value_col ausente, agrupa por group_by, ordena desc/asc, limita, calcula crescimento com pct_change.

****Viz****: se resultado 1 linha → text; se timeseries/growth → line; se group_count e ≤8 grupos → pie; se ranking → bar; senão table.

****Fallback****: classifica erro pela mensagem, monta resposta PT-BR com colunas disponíveis e perguntas sugeridas.

6. Bateria de Perguntas Testadas (11 perguntas, 3 formatos)

| # | Pergunta | Intent | Viz | Resposta (exemplo com sample) |

|---|-----|-----|-----|-----|

- | | | | | |
|----|--|-------------|-------|--|
| 1 | Qual fornecedor recebeu o maior valor no período? | ranking | bar | Fornecedor B SA - R\$ 29.900,00 |
| 2 | Qual produto apresentou o maior volume comprado? | ranking | bar | Notebook Dell - 22 unidades |
| 3 | Qual foi o total gasto em cada mês? | timeseries | line | 2024-01: R\$12.500, 2024-02: R\$8.900... |
| 4 | Quais foram os cinco maiores fornecedores? | ranking | bar | B SA, A LTDA, C ME, D LTDA |
| 5 | Qual categoria apresentou maior crescimento nas compras? | growth | line | Informática cresceu 68% |
| 6 | Qual o valor total das notas fiscais emitidas? | aggregate | text | R\$ 64.101,45 em 6 notas |
| 7 | Quais os 5 emitentes com maior valor total de notas? | ranking | bar | B SA, A LTDA... |
| 8 | Qual UF do destinatário recebeu mais notas fiscais? | group_count | pie | SP - 3 notas (50%) |
| 9 | Qual a quantidade total por NCM? | group_count | pie | 8471: 4 notas, 9403: 2 notas |
| 10 | Qual CFOP mais utilizado? | ranking | bar | 5102 - 5 notas |
| 11 | Liste as notas fiscais com maior valor | list | table | Tabela com 6 linhas ordenadas por vNF |

Todas testadas em `index.html` (client) e `app.py` (Python), com texto + tabela + gráfico Chart.js.

7. Tratamento de Entrada e Perguntas Inválidas

- Upload sem ZIP: "Nenhum CSV encontrado" + exemplo
- Pergunta vazia: alerta
- Coluna não encontrada: Fallback lista colunas disponíveis
- Valor não numérico: converte para 0 e loga warning
- LLM falha (CORS, sem chave): fallback automático para heurística local
- Tabela vazia: "Nenhum resultado"

8. Organização do Projeto

...

/index.html # GitHub Pages - 6 agentes JS + Chart.js

/app.py # Streamlit - orquestrador Python

/src/csv_loader.py # Loader + Schema

/src/models.py # QueryPlan Pydantic

/src/agent.py # Query + Fallback + LangChain

/src/data_engine.py # Executor + Viz

/docs/arquitetura.md # Diagrama e decisões

/docs/relatorio_tecnico.md # Este arquivo

/sample_data/ # ZIP exemplo

/tests/ # Testes unitários

/requirements.txt

/README.md

...

9. Código Organizado e Documentado

- Funções com docstring, logs por agente, sem código duplicado
- `QueryPlan` valida contrato entre agentes
- Nenhum `eval` de código gerado por LLM
- Chaves API via `.env` (não commitado)

10. Limitações Conhecidas e Próximos Passos

- Sem join entre tabelas (cada pergunta sobre uma tabela, requisito futuro: join por chave)
- Heurística PT-BR não é NLU completo (próximo: embeddings + LLM local)
- Dados em memória (não persistidos)
- Próximos: agente só para gráficos (Viz especializado), agente validador de erros, suporte a SQL, cache de consultas

11. Conclusão

A solução unificada entrega documentação robusta do projeto 01 + acessibilidade prática do projeto 02. Mantém 6 agentes bem definidos, expande para multiagentes especializados, usa interface amigável online com recursos avançados (plano JSON, logs, dicionário), consolida Chart.js para visualizações, adota FallbackAgent para robustez, e publica código + GitHub Pages + relatório com 11 perguntas em 3 formatos. Atende todos os requisitos mínimos do Desafio 4 e demonstra como agentes inteligentes

transformam CSV em informação automática.

Licença MIT — I2A2 InsurTechLab 2025/2026